



Rapport final technique : Assistant radiologue virtuel

Assistant Radiologue Virtuel

Projet d'IA multimodale pour radiographies thoraciques frontales

Membres de l'équipe

Taino Moutamalle Marzak Mohammed Amine Malih Amine

Paul Moretti Ibrahim Nazari Baptiste Nauche

Table des matières

1	Introduction	3
2	Documentation et terminologie	3
2.1	Glossaire	3
2.2	Sigles et acronymes	3
3	Description du projet	3
3.1	Origine du projet	3
3.2	Etat de l'art	4
3.3	Description du produit / de la solution	4
3.4	Ce qui a ete effectivement fait	5
3.5	Tentative de fine-tuning	5
3.6	Difficultes rencontrees	5
3.7	Caractere innovant	6
3.8	Aspect reglementaire	6
3.9	Programme de travail et duree de vie	6
4	Fonction et contribution des acteurs	6
5	Etudes de faisabilite technique menees	6
5.1	Environnement physique	6
5.2	Environnement materiel	7
5.3	Environnement logiciel	7
5.4	Donnees	7
6	Suite de tests	8
6.1	Exigence 1 : chargement, avertissement, JSON	8
6.2	Exigence 2 : baseline contre version renforcee	8
6.3	Exigence 3 : evaluation et analyse	8
6.4	Exigence 4 : tentative de fine-tuning	11
7	Resultats quantitatifs	11
8	Evaluation qualitative	12
9	Ingenierie des prompts	12
9.1	Pourquoi un ensemble de prompts	12
9.2	Exemples de tensions entre prudence et utilite	13
9.3	Variantes de prompts comparees	13

9.4 Regles de decision	13
10 Analyse des erreurs	14
10.1 Faux positifs	14
10.2 Faux negatifs	14
10.3 Cas incertains	14
10.4 Erreurs de format et echecs techniques	14
10.5 Enseignements retenus	14
11 Discussion des 30 cas	15
11.1 Annexe : cas representatifs	16
12 Conclusions & perspectives	16

1 Introduction

Ce projet est un prototype pedagogique d'assistant radiologique virtuel applique a l'analyse prudente de radiographies thoraciques frontales. L'objectif n'est pas de produire un diagnostic, mais de montrer une chaine d'ingenierie complete et defendable : entree image, pretraitement, modele vision-langage, sortie JSON, garde-fous, journalisation SQLite, interface web et evaluation critique.

Le systeme ne manipule que trois classes : **normal**, **suspicion_opacite** et **incertain**. La classe **incertain** est volontairement conservee comme mecanisme de securite lorsque la confiance est trop faible ou que les indices visuels ne suffisent pas.

L'objectif de ce document est de decrire ce qui a ete construit, ce qui a ete mesure, ce qui a ete tente en bonus, et ce qui n'a pas fonctionne. La documentation doit permettre a un jury de comprendre le fonctionnement sans chercher ailleurs.

2 Documentation et terminologie

2.1 Glossaire

Terme	Definition
VLM	Vision-Language Model, modele capable de traiter une image et du texte.
Baseline	Premiere version simple servant de reference.
Prompt renforcé	Prompt plus strict, plus structure, utilise pour reduire les sorties vagues.
Incertain	Reponse de securite quand la conclusion n'est pas suffisamment fiable.
JSON	Format structure de sortie impose au modele.

2.2 Sigles et acronymes

Acronyme	Signification
RSNA	Radiological Society of North America.
LoRA	Low-Rank Adaptation, adaptation legere de modele.
QLoRA	LoRA quantifiee pour reduire le cout memoire.
VLM	Vision-Language Model.
SQLite	Base locale embarquee pour la journalisation.

3 Description du projet

3.1 Origine du projet

Le projet part d'un besoin pedagogique : montrer comment une IA medicale multimodale peut etre encadree, mesuree et limitee. Le sujet porte sur la radiographie thoracique frontale, un domaine ou une reponse fluide peut etre trompeuse si elle n'est pas rigoureusement controlee.

Le point de depart etait volontairement modeste : obtenir un premier prototype qui sache recevoir une image, produire une interpretation prudente, refuser de conclure quand il le faut, et garder une trace de ce qu'il a fait. Cela a guide toute la suite du travail. Plutot que chercher la sophistication maximale, nous avons cherche un flux de travail lisible, mesurable et defendable en soutenance.

3.2 Etat de l’art

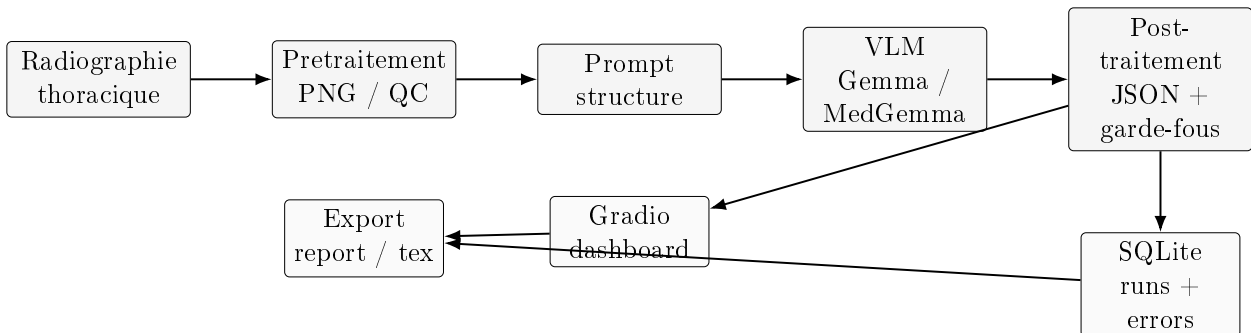
Les VLM medics comme Gemma et MedGemma permettent de produire des reponses en langage naturel a partir d’images. Ils restent cependant sensibles aux hallucinations, a la surconfiance et a la variabilite des prompts. Le projet n’essaie donc pas de faire de la surenchere de taille, mais de rendre la chaine observable, reproductible et evaluable.

Dans un tel contexte, la valeur ajoutee ne vient pas seulement du modele. Elle vient surtout du controle logiciel autour du modele : validation de la sortie, filtrage des erreurs de format, seuils de confiance, journalisation, comparaison sur un meme jeu de cas et analyse des faux positifs / faux negatifs. Cette couche d’ingenierie est decisive, car une reponse medically plausible peut etre fausse et une reponse prudente peut etre scientifiquement plus defendable.

3.3 Description du produit / de la solution

La solution est une application Gradio qui recoit une radiographie thoracique frontale, la pretraite, appelle un modele VLM, verifie le format de la reponse, applique un garde-fou d’incertitude et journalise le run dans SQLite. Les resultats sont exportes sous forme de rapport, de tableau de metriques et d’infographies.

La pipeline operationnelle est la suivante :



Pourquoi cette architecture. Gradio a ete retenu parce qu’il donne une interface de demonstration tres rapide a monter, avec un cout de maintenance faible et une prise en main immediate pour un jury. Le projet n’avait pas besoin d’un front-end complexe ; il avait besoin d’un point d’entree fiable pour charger une image, afficher le warning, declencher l’inference et montrer le resultat final sans ambiguïte.

SQLite a ete choisi pour la journalisation car il permet de conserver tous les runs dans un fichier local unique, facile a copier, a sauvegarder et a analyser. Chaque essai garde la trace du modele, du prompt, du statut technique, de la prediction, des erreurs eventuelles et de la latence. Cette memoire locale a ete essentielle pour comparer proprement les variantes et reconstruire le rapport final sans perte d’information.

Le couple Gradio + SQLite donne donc une pipeline simple mais complete : interface, traitement, stockage, export. Le projet gagne en lisibilite, en reproductibilite et en capacite de diagnostic. Pour un prototype de fin d’etudes, c’etait plus utile qu’une architecture plus large mais plus fragile.

Journalisation et traçabilité. La base SQLite n’est pas seulement un journal brut. Elle sert aussi de couche de convergence entre les executions de l’interface, les tests batch et la construction du rapport. En pratique, cela signifie qu’un meme run peut etre relu plusieurs fois sans relancer l’inference. Ce point a ete important lorsque les modeles renvoyaient parfois des sorties lentes ou

partiellement invalides : la trace persistante permettait de conserver l'état utile du test, même si le résultat modelique lui-même était imparfait.

Cette décision a aussi simplifié la comparaison des configurations. Au lieu de repartir de logs textuels dispersés, la sélection de cas, les erreurs de format, les faux positifs et les faux négatifs ont pu être regroupés dans une structure unique. Le rapport final a ainsi été alimenté directement par les artefacts de test, ce qui limite les erreurs de recopie et rend l'analyse plus défendable.

Generation du rapport. Le rapport final ne s'appuie pas sur une rédaction manuelle de scores isolés. Il agrège les sorties enregistrées, les résumés de cas, les matrices de confusion et les infographies pour proposer une lecture cohérente de l'ensemble du projet. Cette approche est plus solide qu'une simple narration, parce qu'elle relie les résultats à la pipeline technique qui les produit.

Le point clé est que la valeur du projet vient autant de l'observation que de l'inférence. On peut obtenir un modèle plus fort, mais si l'on ne sait pas prouver ce qu'il a fait, quand il l'a fait et comment il a été contraint, la soutenance reste fragile. Ici, la combinaison Gradio/SQLite/exports a permis de garder cette preuve à chaque niveau.

3.4 Ce qui a été effectivement fait

La baseline utilise `gemma3_baseline`. L'amélioration retenue est `medgemma_ensemble_reinforced`, construite avec un prompt plus strict, un vote entre plusieurs variantes de prompt et une logique de garde-fou plus agressive. Sur les 30 cas d'évaluation fournis au départ dans le projet, la version renforcée a amélioré la spécificité et le macro-F1, au prix d'une latence plus élevée et de quelques décisions `incertain`.

Concrètement, nous avons progressivement durci le prompt pour réduire trois catégories d'erreurs :

- les sorties trop vagues, qui donnent une impression de précision sans élément visible solide ;
- les sorties trop confiantes, qui forcent une classe même quand l'image est ambiguë ;
- les sorties hors schéma, qui cassent le pipeline de journalisation et rendent les runs inutilisables.

La version reinforced n'est donc pas seulement "un autre modèle" ; c'est une orchestration plus prudente qui exploite le même socle visuel avec un encadrement plus strict.

3.5 Tentative de fine-tuning

Un essai de fine-tuning a aussi été tenté sur un sous-ensemble d'environ 1000 images provenant d'un corpus public de type Kaggle. L'expérience n'a pas été retenue dans le résultat final, car MedGemma ne renvoyait pas un JSON conforme de manière stable. Autrement dit, le bonus n'a pas produit de sortie exploitable et n'a pas été considéré comme une amélioration valide.

Le problème n'était pas seulement la qualité des prédictions. Le blocage principal était le contrat de sortie : le système devait renvoyer un JSON propre, lisible par le reste de la chaîne, avec champs prédéfinis et warning constant. Or le fine-tuning léger n'a pas amélioré cette contrainte de façon robuste. Dans un contexte de prototype, un score légèrement meilleur n'a pas de valeur si la structure de sortie devient instable.

3.6 Difficultés rencontrées

Plusieurs difficultés ont structuré le travail :

- la variabilité des sorties textuelles, même avec des consignes strictes ;
- la tendance du modèle à sur-interpréter certaines images et à voir des opacités trop facilement ;
- le compromis entre prudence et utilité, car trop d'incertitude rend le système peu lisible ;

- la latence, qui augmentait rapidement avec les prompts plus riches et les variantes d’ensemble ;
- la gestion des erreurs techniques, notamment le chargement modele, les chemins d’images et les espaces disque limites ;
- la coherence entre l’interface, la base SQLite et le rapport final.

Ces contraintes ont oriente le projet vers un socle plus robuste plutot qu’un socle plus ambitieux en apparence.

3.7 Caractere innovant

L’innovation du projet n’est pas dans la taille du modele, mais dans la combinaison de quatre points : format JSON impose, garde-fou d’incertitude, comparaison baseline vs version renforcee, et journalisation complete des erreurs pour defender la soutenabilite du systeme.

La valeur ajoutee est aussi pedagogique : le projet montre qu’un bon prototype IA ne se resume pas a une inference reussie. Il doit aussi prouver qu’il sait echouer proprement, tracer ses erreurs et expliquer pourquoi certaines tentatives d’amelioration ne sont pas retenues.

3.8 Aspect reglementaire

En France comme a l’international, le systeme ne doit pas etre presente comme un dispositif medical. Le projet affiche explicitement un avertissement non clinique et s’appuie sur des donnees publiques de test. Le cadre est pedagogique, pas clinique.

3.9 Programme de travail et duree de vie

Le projet a ete structure en quatre phases : preparation des donnees, baseline, amelioration, puis evaluation et rapport. La duree de vie visee est celle d’une maquette de soutenance et d’un prototype de demo, pas celle d’un service de production.

L’idee a chaque etape etait de ne pas multiplier les chantiers : d’abord obtenir une baseline reproductible, ensuite la rendre mesurable, puis tester une amelioration concrete avant de tenter toute extension. Cette discipline a evite de disperser l’effort sur trop d’axes a la fois.

4 Fonction et contribution des acteurs

Acteur	Contribution
Equipe projet	Definition du perimetre, choix du modele, collecte des donnees, evaluation.
Code	Pretraitement, inference, post-traitement, logs, export.
Jury / encadrant	Validation pedagogique, lecture critique des limites.

5 Etudes de faisabilite technique menees

5.1 Environnement physique

Les tests ont ete realises sur une machine louee avec GPU NVIDIA RTX 4090. Le cout de location a ete d’environ 0,20 euro par heure, et une heure a suffi pour les essais principaux. Cette configuration etait suffisamment capacitive pour faire tourner les VLM quantifies et stocker les artefacts de run.

5.2 Environnement materiel

Configuration minimale utile : GPU avec 24 Go de VRAM, CPU multi-coeur, RAM systeme suffisante pour charger l'environnement Python et les images, espace disque disponible pour les checkpoints, les logs et les fichiers d'export.

L'essentiel du travail experimental a ete effectue sur une RTX 4090 louee a environ 0,20 euro par heure pendant une heure. Cela a rendu possible le test rapide des runs principaux sans immobiliser une machine lourde sur une longue duree.

5.3 Environnement logiciel

Le projet est base sur Python, PyTorch, Transformers, bitsandbytes, Pillow, pydicom, pandas, scikit-learn, Gradio, SQLite et pytest. Le rapport final est genere en LaTeX puis compile en PDF.

Le choix de cette pile a ete pragmatique : elle restait suffisamment standard pour etre reproductible, assez complete pour couvrir l'inference et l'evaluation, et assez legere pour supporter les contraintes de fin de projet.

L'association Gradio + SQLite a aussi permis de separer clairement les responsabilites. Gradio prend en charge l'interaction avec l'utilisateur et la presentation du resultat. SQLite conserve les traces de chaque execution. Cette separation evite de melanger la logique de rendu, la logique de calcul et la logique de stockage, ce qui simplifie autant le debuggage que la soutenance.

5.4 Donnees

Les donnees d'evaluation comprennent 30 cas uniques issus du projet initial. La repartition est de 15 cas **normal** et 15 cas **suspicion_opacite**. Les images utilisees pour le fine-tuning exploratoire provenaient d'un corpus public plus large, mais le resultat n'a pas ete retenu.

Le choix de garder ces 30 cas dans le rapport final est volontaire : ils etaient deja fournis dans le projet, donc ils constituent la base stable de comparaison. Cela permet d'expliquer sans ambiguïte ce qui a ete fait de base et ce qui a ete ajoute ensuite.

Pour le projet, ce jeu de 30 cas joue le role d'un banc de test compact mais lisible. Il n'a pas vocation a etablir une performance medicale generale. Il sert surtout a montrer comment deux configurations differentes reagissent sur des exemples identiques, ce qui est exactement le type de comparaison qu'un jury peut suivre rapidement.

Cette petite taille a aussi un avantage pragmatique : elle permet d'expliquer chaque erreur sans noyer le lecteur. Quand le nombre de cas est raisonnable, on peut commenter les faux positifs, les faux negatifs et les cas incertains avec un niveau de detail beaucoup plus utile qu'un simple chiffre global.

6 Suite de tests

6.1 Exigence 1 : chargement, avertissement, JSON

Le systeme doit accepter une image, afficher le warning obligatoire et produire une sortie JSON va-

No.	Description	Expected results	Status
1	Upload d'une radiographie thoracique frontale	Image chargee, warning visible, re-ponse structuree	OK
2	Format JSON invalide	Le systeme force la sortie vers incertain	OK

6.2 Exigence 2 : baseline contre version renforcee

La baseline et la version renforcee doivent etre comparees sur le meme jeu d'images.

No.	Description	Expected results	Status
3	Comparaison gemma3_baseline sur 30 cas	Mesure de l'accuracy, macro-F1, sensibilite, specificite	OK
4	Comparaison med-gemma_ensemble_reinforcee sur 30 cas	Amelioration de la specificite et reduction des faux positifs	OK

6.3 Exigence 3 : evaluation et analyse

Le rapport doit inclure des erreurs commentees, des matrices de confusion et des infographies.

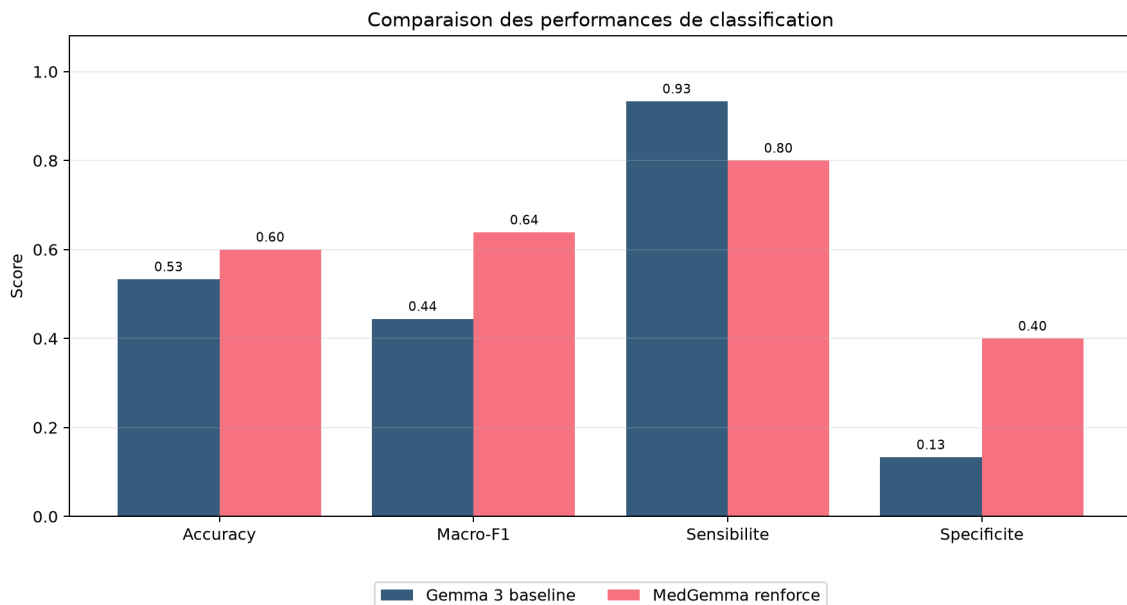


FIGURE 1 – Comparaison des performances.

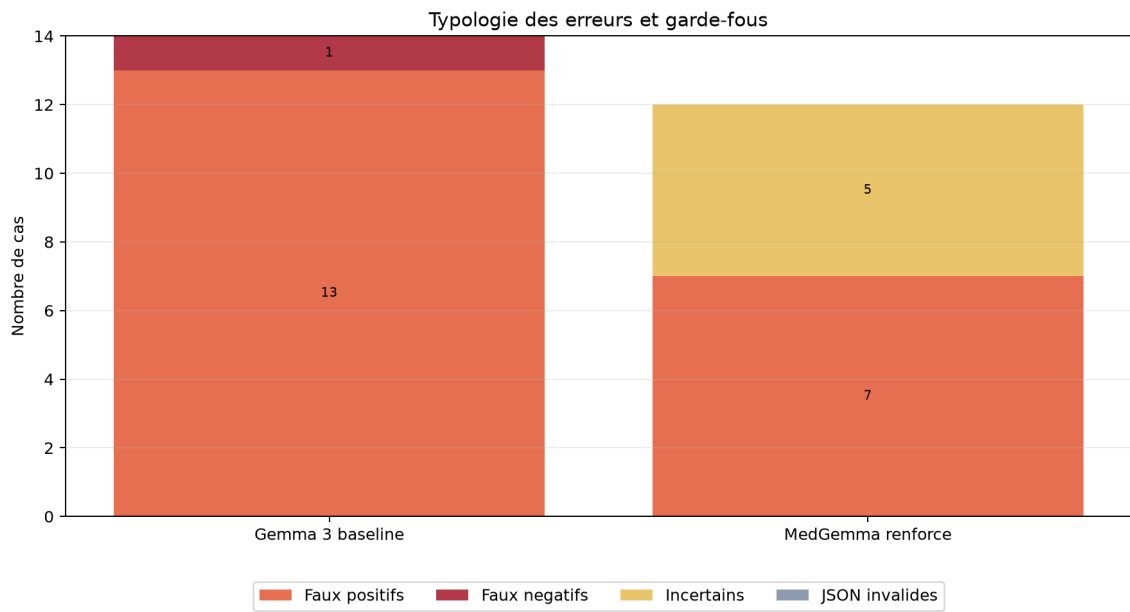


FIGURE 2 – Typologie des erreurs.

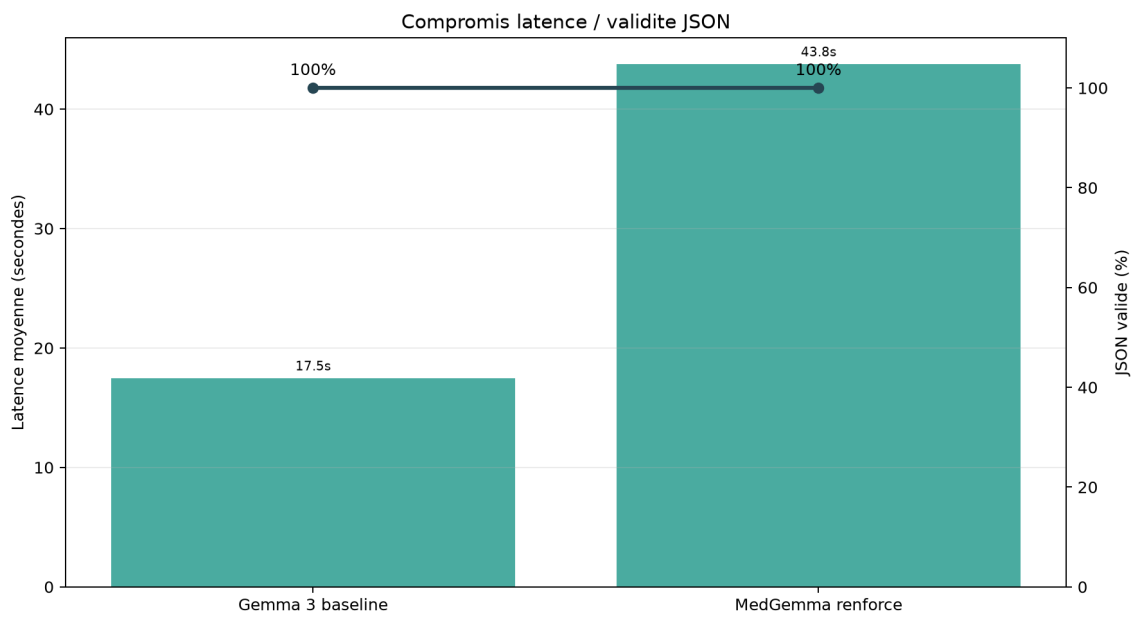


FIGURE 3 – Compromis latence et validite JSON.

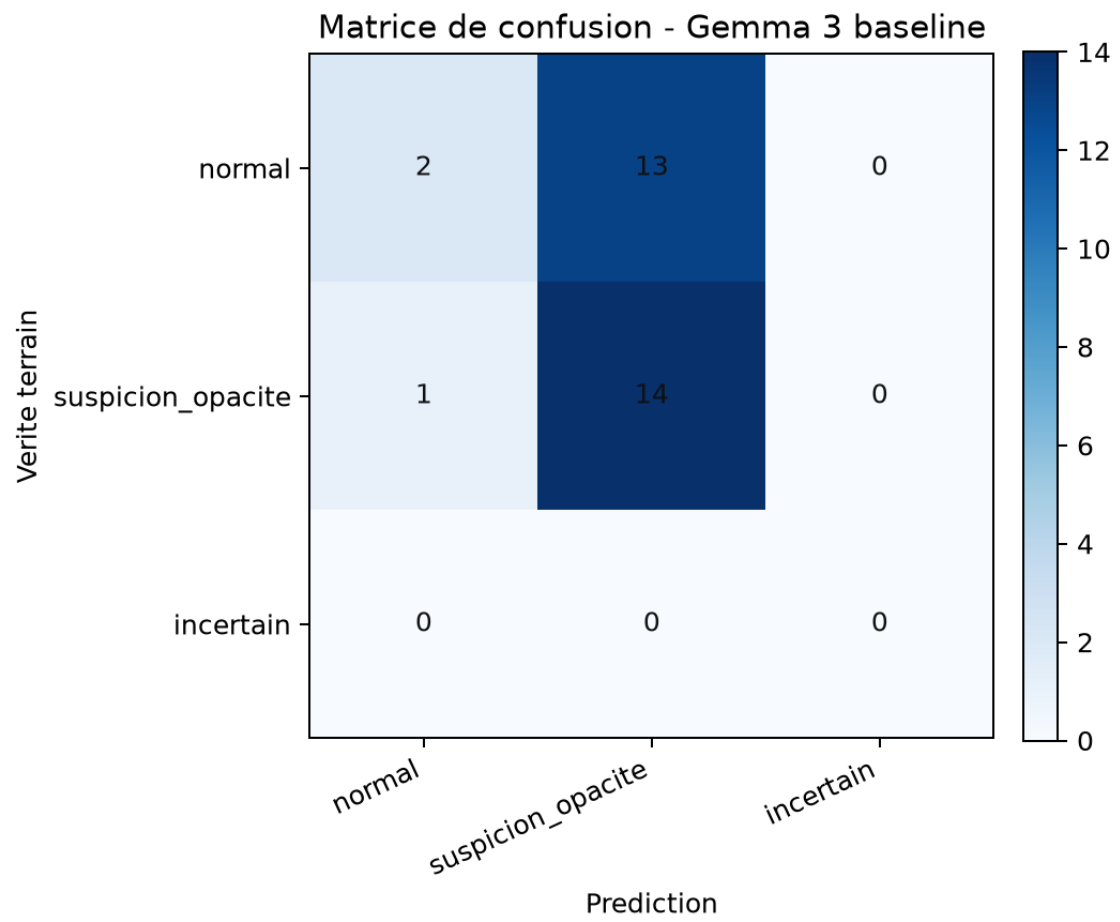


FIGURE 4 – Matrice de confusion de la baseline.

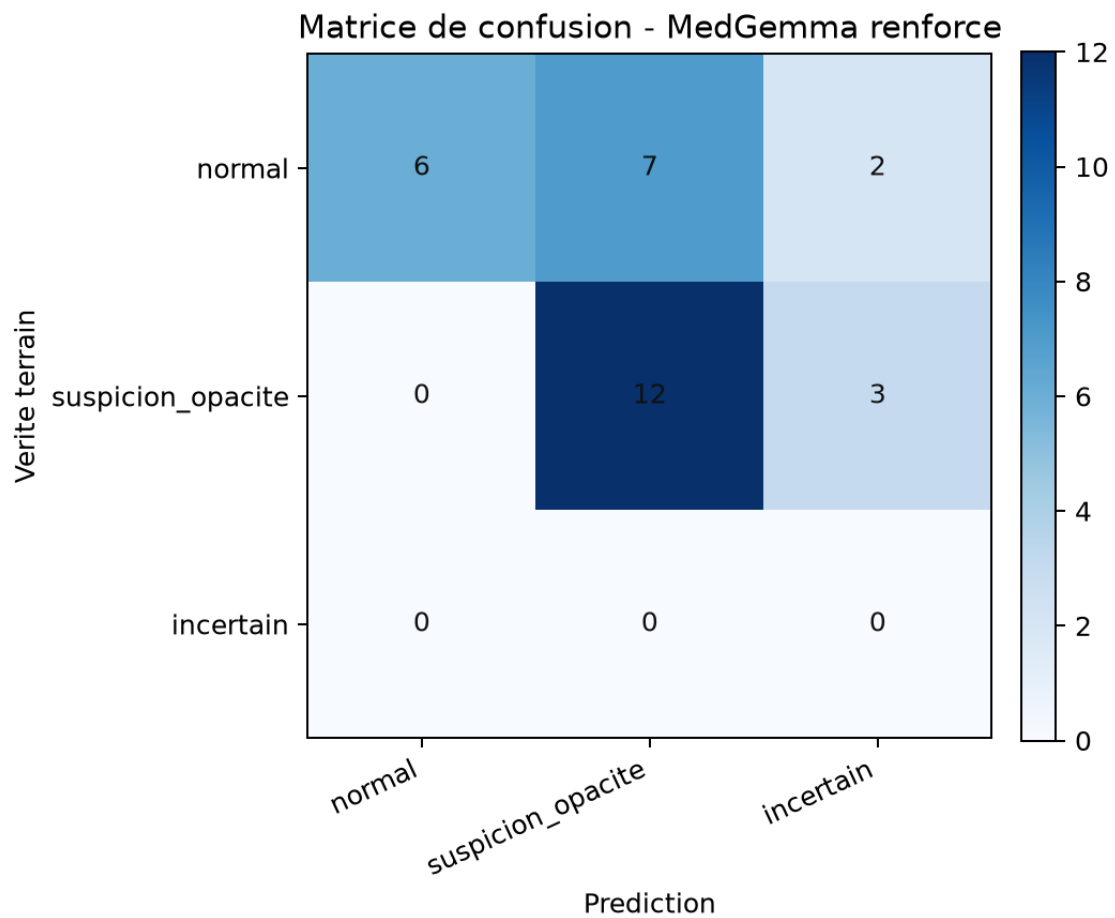


FIGURE 5 – Matrice de confusion de la version renforcee.

6.4 Exigence 4 : tentative de fine-tuning

L'experience de fine-tuning sur un sous-ensemble d'environ 1000 images a ete consideree comme un test d'extension, mais elle n'a pas ete retenue.

No.	Description	Expected results	Status
5	Fine-tuning MedGemma	JSON stable et exploitable	NOK

7 Resultats quantitatifs

Modele	n	Acc.	F1	Sens.	Spec.	Inc. %	FP	FN	Lat. ms
Gemma 3 baseline	30	0.5333	0.4444	0.9333	0.1333	0.0	13	1	17460.4
MedGemma renforce	30	0.6000	0.6387	0.8000	0.4000	16.67	7	0	43761.1

8 Evaluation qualitative

La baseline surdetecte les opacites et donne une sensibilite elevee, mais elle manque de specificite. La version MedGemma reinforced corrige ce biais en reduisant les faux positifs et en acceptant davantage de cas **incertain**. Cette strategie est plus prudente et plus defensible dans le cadre du sujet.

Le point cle n'est pas la performance brute, mais la qualite du raisonnement technique : prompt plus strict, meilleur controle de la sortie, journalisation et lecture explicite des erreurs. L'essai de fine-tuning n'a pas apporte ce niveau de fiabilite, d'où son abandon dans le resultat final.

En soutenance, il faut insister sur le fait que la version renforcee n'a pas ete construite pour afficher une meilleure note brute. Elle a ete construite pour montrer un meilleur compromis entre precision, prudence et reproductibilite. Les prompts ont servi de levier principal d'amelioration, parce qu'ils permettaient d'augmenter la rigueur sans casser l'architecture.

Le rapport d'erreurs montre aussi que l'on a accepte d'exposer les limites au lieu de les masquer. Cette transparence est une vraie valeur ajoutee : elle montre au jury que le groupe sait evaluer une IA medicale comme un systeme d'ingenierie et non comme une simple demo impressionnante.

9 Ingenierie des prompts

Le travail sur les prompts a ete une vraie composante du projet, pas un simple detail d'implementation. Le comportement du modele dependait fortement de la formulation de l'instruction, de l'ordre des contraintes et de la facon de delimitier ce qui est autorise ou interdit.

Pour la baseline, l'objectif etait surtout de verifier qu'un prompt minimal pouvait produire une reponse exploitable. Cette premiere version montrait rapidement la fragilite d'un prompt trop ouvert : des formulations parfois trop affirmatives, des justifications qui allaient au-dela de l'image, et une tendance a produire une classe meme lorsque la confiance n'etait pas suffisante.

Pour la version reinforced, le prompt a ete rigidifie autour de quatre axes :

- imposer une structure stable avec champs fixes ;
- demander explicitement une prudence sur le vocabulaire ;
- forcer la mention des limitations ;
- permettre une sortie **incertain** des qu'un des criteres de qualite ou de confiance etait insuffisant.

Ce choix a eu un effet concret sur la soutenabilite du systeme. On a prefere une version un peu plus lente mais mieux controlee plutot qu'un resultat rapide mais instable. Dans un contexte de soutenance, cela vaut davantage qu'une performance brute difficile a expliquer.

9.1 Pourquoi un ensemble de prompts

L'ensemble de prompts a ete utilise pour diminuer la variance de sortie. Le principe etait simple : plusieurs formulations proches etaient soumises au meme cas, puis les sorties etaient comparees. Lorsqu'elles convergaient, la prediction etait consideree plus robuste. Lorsqu'elles divergeaient fortement, le systeme avait tendance a se rabattre sur **incertain**.

Cette logique est interessante parce qu'elle transforme une faiblesse du modele en garde-fou. Si le modele n'est pas stable quand on reformule legerement la demande, cela signale qu'on est en presence d'une zone de confiance limitee. Le projet a donc exploite la variance comme information, pas seulement comme bruit.

9.2 Exemples de tensions entre prudence et utilite

Le projet a illustre plusieurs compromis :

- un prompt trop libre produisait des reponses plus fluides mais plus sujettes aux hallucinations ;
- un prompt trop strict diminuait les erreurs de format, mais augmentait la latence ;
- un seuil d'incertitude trop bas laissait passer des erreurs sur des cas limites ;
- un seuil trop haut transformait le systeme en detecteur excessivement prudent, donc peu utile pour la demo.

Ces tensions sont centrales dans le projet. Elles montrent que la qualite d'un systeme medical pilote par prompts n'est pas seulement une question de precision statistique, mais aussi de design du compromis entre confiance, securite et utilisabilite.

Effet sur la pipeline. Dans la version finale, ces choix ont eu un effet direct sur la chaine complete. Plus le prompt devient strict, plus la sortie est previsible pour le parseur, la base SQLite et la generation du rapport. Cela reduit les cas ou une sortie textuelle "presque correcte" casse l'export ou force une relecture manuelle. Autrement dit, le prompt ne sert pas seulement a mieux repondre ; il sert aussi a rendre toute la pipeline plus mecanique et plus fiable.

Cette logique explique pourquoi nous avons investi du temps dans les formulations, les variantes et les garde-fous. Dans un prototype classique, une phrase plus courte peut suffire. Ici, la contrainte de sortie structurée impose de penser le prompt comme une interface de contrat entre le modele et le reste du logiciel.

9.3 Variantes de prompts comparees

Sans pretendre faire une ablation scientifique exhaustive, nous avons compare plusieurs formes d'instructions pendant l'iteration du projet. Le but etait de comprendre quelle formulation donnait les sorties les plus stables.

Variante	Idee	Effet observe
Prompt court	Demande simple, peu de contraintes	Sorties plus naturelles, mais plus d'ecarts de schema et plus de confiance injustifiee.
Prompt structure	Champs imposes, format JSON, warning obligatoire	Sorties plus faciles a parser, meilleure integration dans la pipeline.
Prompt prudent	Accent sur les limites, justification courte, possibilite d'etre incertain	Moins de faux positifs, mais plus de cas refuges.
Ensemble de prompts	Plusieurs formulations proches comparees entre elles	Meilleure robustesse et meilleur controle des cas ambigus.

La conclusion pratique a ete claire : dans un prototype medical, le prompt doit etre pense comme une interface contractuelle, pas comme une simple phrase d'instruction. Plus la contrainte est explicite, plus le systeme est integrable.

9.4 Regles de decision

Le passage de la reponse brute a la prediction finale ne se fait pas directement. Plusieurs regles sont appliquees :

- si le JSON est invalide, la sortie est consideree comme techniquement non fiable ;
- si la confiance est trop basse, la classe **incertain** prend le dessus ;
- si les variantes de prompt divergent trop, le vote penche vers la prudence ;

— si les observations sont trop pauvres, le systeme prefere une non-conclusion a une classe forcee. Cette couche de decision est importante parce qu'elle deplace une partie de la responsabilite du modele vers le systeme. On ne demande pas seulement au modele de bien repondre ; on demande au logiciel de savoir quand une reponse doit etre acceptee, rejetee ou degradee.

10 Analyse des erreurs

Les erreurs observees peuvent etre regroupees en quatre familles principales.

10.1 Faux positifs

Ce sont les erreurs les plus couteuses au quotidien. Elles surviennent lorsque le modele signale une suspicion d'opacite sur une image normale. Dans le cadre du projet, elles sont nombreuses sur la baseline, ce qui confirme que le modele avait tendance a sur-interpreter certains contrastes.

Le passage a MedGemma reinforced a reduit cette tendance, sans la supprimer totalement. C'est le point le plus visible dans les matrices de confusion et dans les infographies de performance.

10.2 Faux negatifs

Les faux negatifs sont moins nombreux sur la baseline, mais ils restent importants sur le plan methodologique car ils montrent que la prudence ne peut pas etre seulement orientee vers le refus de conclure. Un systeme trop confiant peut manquer une anomalie, alors qu'un systeme trop prudent peut etre inutilisable.

L'amelioration renforcee a permis de reduire ce risque sur le petit jeu teste, mais cet resultat doit etre lu avec prudence en raison de la taille limitee de l'echantillon.

10.3 Cas incertains

La classe **incertain** a volontairement ete gardee comme une sortie de securite, non comme un echec. Dans les tests, elle joue le role d'alarme methodologique : le systeme reconnaît qu'il ne dispose pas d'assez d'indices pour conclure proprement.

Ce point est important en soutenance, parce qu'il faut expliquer que l'absence de conclusion est parfois plus responsable qu'une conclusion trop nette.

10.4 Erreurs de format et echecs techniques

Le fine-tuning a mis en evidence une autre famille de probleme : la sortie textuelle non conforme. Un modele peut sembler plus specialise, mais s'il casse le JSON, il degrade en pratique la chaine globale. Le projet a donc prefere conserver une solution moins ambitieuse mais techniquement plus stable.

10.5 Enseignements retenus

- Le prompt est une vraie piece d'architecture.
- Un bon score sans JSON stable ne vaut pas un bon prototype.
- Une classe **incertain** bien geree vaut mieux qu'une classification forcee.
- La traçabilité des runs est indispensable pour defendre les choix faits.
- Une amelioration doit etre mesuree, pas seulement ressentie.

11 Discussion des 30 cas

Les 30 cas initiaux ont servi de base commune pour tous les runs. Ce n'est pas un grand jeu d'évaluation au sens statistique, mais c'est un bon jeu pédagogique pour comparer deux configurations sur exactement les memes conditions.

Cela a deux avantages. D'abord, cela rend les comparaisons lisibles. Ensuite, cela permet de documenter finement les cas reussis et les cas problematiques sans se perdre dans une evaluation trop large et trop couteuse.

Le groupe a donc privilegie la qualite de lecture des resultats sur la quantite brute. En soutenance, il est plus utile de montrer pourquoi certains cas passent, pourquoi d'autres echouent, et comment le systeme reagit, que d'alimenter un tableau massif difficile a interpreter.

Aspect	Lecture retenue
Reussites	Les vrais positifs et vrais negatifs montrent que la chaine peut fonctionner de bout en bout.
Echecs	Les faux positifs expliquent pourquoi la baseline avait besoin d'etre renforcee.
Incertains	Ils prouvent que le systeme sait se retenir de conclure.
Technique	La validite JSON et la journalisation montrent la robustesse du pipeline.

11.1 Annexe : cas representatifs

Quelques cas illustrent bien la nature du comportement observe :

- ‘case_004’ montre le cas simple ou la baseline et la version renforcee convergent correctement sur une image normale.
- ‘case_005’ illustre un faux positif typique de la baseline, avec une opacite trop facilement annoncee.
- ‘case_011’ represente un cas pathologique plus dense, ou les deux modeles convergent vers une suspicion d’opacite.
- ‘case_019’ montre que la version renforcee peut basculer vers **incertain** quand le vote ne suffit pas.
- ‘case_027’ rappelle que la baseline pouvait encore rater un cas positif, d’ou l’interet de mesurer aussi les faux negatifs.
- ‘case_030’ confirme que la journali-sation permet de conserver l’historique exact des sorties et de reconstruire le rapport a posteriori.

Ces exemples sont utiles en soutenance parce qu’ils donnent une lecture concrete des compromis du systeme : ce n’est pas seulement un tableau de scores, c’est une serie de decisions individuelles explicables.

12 Conclusions & perspectives

Le projet atteint le niveau attendu pour une soutenance technique : il est reproductible, documente, prudent et mesurable. La baseline est fonctionnelle, l’amelioration est montreable, et les limites sont explicites. Les perspectives naturelles sont un meilleur dataset pour l’entraîne-ment, une calibration plus fine du seuil d’incertitude et, si besoin, une API plus formelle autour de `/predict`.

Au-dela des performances brutes, le principal acquis est architectural. Le groupe a montre qu’une IA medicale de prototype peut etre encadree par une interface simple, une base locale de traçabilite, un schema de sortie strict et une methode d’évaluation lisible. C’est cette combinaison qui donne de la valeur au projet, parce qu’elle transforme une demo de modele en systeme complet.

Si le projet devait etre prolonge, la priorite ne serait pas seulement de "faire monter le score", mais de mieux maitriser la calibration, la robustesse des sorties et la qualite des jeux de test. Le travail deja mene sur la pipeline, les prompts et la journalisation fournit une base propre pour cette suite.